

In many languages, the terms *property* and *attribute* are used synonymously, but in Python these words have distinct meanings. An attribute, explained in “Variable vs. Attribute” on page 124, is a name associated with an object. Attributes include the object’s member variables and methods.

Other languages, such as Java, have `getter` and `setter` methods for classes. Instead of being able to directly assign an attribute a (potentially invalid) value, a program must call the `setter` method for that attribute. The code inside the `setter` method can ensure that the member variable only has a valid value assigned to it. The `getter` method reads an attribute’s value. If an attribute is named, say, `accountBalance`, the `setter` and `getter` methods are usually named `setAccountBalance()` and `getAccountBalance()`, respectively.

In Python, *properties* allow programmers to use getters and setters with much cleaner syntax. Chapter 17 explores Python properties in more detail.

Bytecode vs. Machine Code

Source code is compiled into a form of instructions called *machine code* that the CPU directly carries out. Machine code is composed of instructions from the CPU’s *instruction set*, the computer’s built-in set of commands. A compiled program composed of machine code is called a *binary*. A venerable language like C has compiler software that can compile C source code into binaries for almost every CPU available. But if a language such as Python wants to run on the same set of CPUs, a large amount of work would have to go into writing Python compilers for each of them.

There is another way of turning source code into machine-usable code. Instead of creating machine code that is carried out directly by CPU hardware, you could create *bytecode*. Also called *portable code* or *p-code*, bytecode is carried out by a software interpreter program instead of directly by the CPU. Python bytecode is composed of instructions from an instruction set, although no real-world hardware CPU carries out these instructions. Instead, the software interpreter executes the bytecode. Python bytecode is stored in the `.pyc` files you sometimes see alongside your `.py` source files. The CPython interpreter, which is written in C, can compile Python source code into Python bytecode and then carry out the instructions. (The same goes for the Java Virtual Machine [JVM] software,